Plainspeak: A Programming Language That Reads Like English

Isaac Evans, Thanh Nguyen Pham, Hitarth Patel, Zac Cappello | Adelaide University | LMTE Assignment 2a

Introduction

Plainspeak is a high level, statically typed, compiled programming language whose source code reads as short English instructions. Instead of symbols such as =,+,== and for(), a program says “set x to 5, add 5 to x” or “if x is greater than 5”. The only punctuation left is curly braces (blocks), parentheses (grouping and call arguments), double quotes (strings) and square brackets. Here our ain was to help beginners understand key concepts, while still covering what a real coding language needs which is typed variables, constants, expressions, input and output, if and switch, iteration, functions, fixed capacity lists, block scope, structs, enums, and classes with single inheritance.

The compiler is written in C++. It reads a .psk source file, checks it, and emits LLVM IR, which LLVM optimises and compiles to a native executable. Using LLVM gives us a mature optimiser and many targets without writing a back end, so our effort goes into the language front end and its semantics.

Idiosyncrasies, and how Plainspeak differs from existing languages:
English coding languages aren't new, and here Plainspeak holds similarities with the language like AppleScript and COBOL, so the differences that can be seen between out language and others can be seen structurally, not through vocabulary.

- **One phrase per construct**: AppleScript accepts several synonyms for one operation (is, equals, is equal to). Plainspeak has exactly one spelling of each, keeping the grammar small and parseable
- **Explicit static typing, declared inline**: “Initialise int x” then “set x to 5” declaring and assigning are always separate, visible steps. AppleScript is dynamically typed where COBOL declares types in picture clauses in a separate data division
- **Braces and newlines, not terminators**: COBOL ends sentences with a full stop (or end if) and AppleScript with end if. Braces make nesting explicit and remove the dangling else problem.
- **Verbs for statements, operators for expressions**: Statements that change a variable are verbs (multiply 5 by x) expressions that produce a value use word “operators (x times 5)”. The two never mix, so we write “return x plus 5”, not “return add 5 to x”
- **Stack only memory model**: Lists have a fixed capacity and every value lives on the stack. There are no pointers, heap, garbage collector or manual free. Overruns become readable run time errors, not undefined behaviour

Syntactic Description

In our language, there are four concepts that helped us decide our final syntax:

- **Read aloud rule**: Every statement has to start with a keyword naming the action that will be done (e.g. set, say, add, put, call, return) so the first token of a line identifies the kind of statement that needs to happen
- **Declare then set**: “initialize <type> name” creates a variable holding its types default “set name to value” changes it. Every name has one static type, which is checked before the program runs
- **Words not symbols**: Operators are phrases, a multi word phrase such as “is greater than or equal to” is one token
- **Structure from braces**: Blocks are {..} and nest to any depth, a statement ends at the end of its line, the file itself is the outermost block.

Basic Syntax

Expressions

Expressions combine values, variables, calls and word operators, with parentheses for grouping. Operators of one level apply left to right.

1 (tightest)	2	3	4	5(loosest)
parentheses, call, item N of, length of	not	times, divided by, mod	is equal to, is not equal to, is greater than, is less than, or equal to	and, then or

Variables, constants and literals

Example	Description
initialize int age set age to 19 initialize string name set name to "Zac" initialize float price set price to 4.50	Types are integer, float, string and bool. Initializing a variable and assignment has to be done across two operations

initialize bool done fix float pi to 3.14159	

Input, output and arithmetic

Example	Description
say name say "Hello " plus name ask "Enter your age: " into age add 5 to x subtract 5 from x multiply 5 by x divide x by 5 mod x by 5	say prints any expression followed by a newline. Ask prints its prompt, reads a line, and converts it to the type of the (already declared) target variable. The five arithmetic statements update the named variable in place. The target must be a non constant numeric variable. Operands may be any expression of the right type

Selection

Example	Description

if x is greater than 5 {
  say "Large"
}
otherwise if x is equal to 5 {
  say "Five"
}
otherwise {
  say "Small"
}
check option {
  when 1 { say "One" }
  when 2 { say "Two" }
  otherwise {
  say "Invalid"
  }
}
	Conditions are bool expressions built from the six comparisons and and, or, not. Otherwise chains are allowed, and otherwise is optional.
	Check is the switch statement. It compares an integer, string or enum value with each when value in order, with no fall through, and runs otherwise if none match

Iteration

Example	Description

repeat for 10 { say "Hi" }
while health is greater than 0 {
  subtract 10 from health
}
for each v in nums { say v }
stop
skip
	Repeat for N evaluates N once, then runs the block that many times. While tests its condition before every pass. For each visits each item of a list in order, its loop variable exists only inside the loop. Stop and skip are legal only inside a loop

Lists (multi element variables)

Example	Description
initialize integer list arr 20 set arr to [1, 2, 3] remove top arr set item 2 of arr to 90 say item 1 of arr say length of arr say capacity of arr	A list is one dimensional, holds one element type (integer, float, string or bool) and has a fixed capacity given at declaration, so it can live on the stack. It starts empty. Put adds at the end and remove top removes the last item. Indexing starts at 1, so item 1 of arr is the first item. A bad index, put on a full list, or remove top on an empty list is a run time error

Functions

Example	Description

initialize function addFive(integer x) returning integer {
  return x plus 5
}
initialize function greet(string who) returning nothing {
  say "Hello " plus who
}
call greet("Zac")
set y to call addFive(y)
	A function declares typed parameters and one return type (nothing for none). It returns a single value with return, and every path of a value returning function must return. Functions are invoked with call, either as a statement or inside an expression. Arguments are passed by value, recursion is supported, and a function may be used before the line that declares it

Scope

Plainspeak is block scoped and the file is the outermost block. A name is visible from its declaration to the closing brace of its block, including nested blocks and function bodies, and a name cannot be redeclared while an earlier declaration is visible (no shadowing). Functions, structs and classes may be used before their declaration, variables may not. A loop variable is visible only in its loop.

Advanced Syntax

Structs & Enums

Example	Description

Define struct Point {
  integer x
  float y
}
initialize Point p
set x of p to 3
Define enum Colour {red, green, blue}
	A struct groups typed fields, read and written with **x of p** .
	
	An enum defines names integer constants numbered from 0. (red is 0)

Classes

Example	Description

define class Animal {
  public initialize string name
  public define function speak() returning nothing {
  say "Animal Noise"
  }
 	
define class Dog inherits Animal {
  private initialize integer age
  public define constructor {string who, integer years} {
  set name to who
  set age to years
  }
Initialize Dog pike with ("Pike", 1)
call bark() from pike
say name of pike
	A class adds methods and encapsulation. Every member is marked public or private, and private members are usable only inside their own class. Objects are created with a constructor call and are live on the stack "initialize Dog pike with ("Pike", 1)". Methods are called with "call bark() from pike" and fields read with "name of pike". "inherits" gives a class all public members of exactly one parent and constructors are not inherited, so Dog can also "call speak() from pike". Method calls are resolved at compile time, so there is deliberately no method overriding or polymorphism. This keeps the semantic rules and code generation simple

Garbage Collection

If time permits, our goal is to create a garbage collector. This would include the basic functions such as making sure the memory associated with data that isn't being referenced is freed. Whilst this isn't our main goal, it would provide a good opportunity for us to extend our programming language.

Compiler Structure

Plainspeak follows a standard compiler structure used by most compiler language (Figure 1). It is written in c++23 with LLVM's c++ API . Our language uses a .psk extension which users will have to use when typing in our language

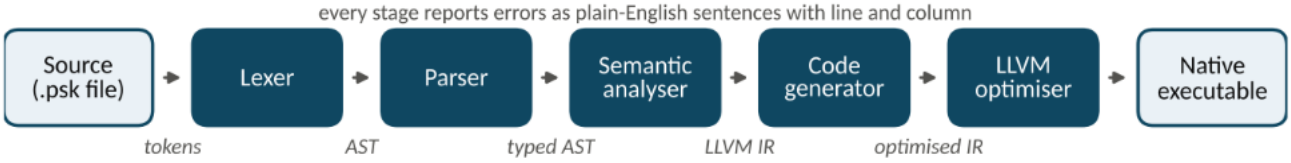


Figure 1: Compiler structure

Tokenisation / Lexical Analysis

The lexer is the first stage of compilation. It scans the *.psk source once and produces a token buffer, where each token stores its type, value, and line and column, ready for the parser to consume. The lexer never fails even if it meets text it cannot recognise, it gives an error token and carries on, so a token buffer is always returned. Following are the rules the lexer must perform before it is ready to hand over token buffer to parser:

- **Multi word key words** (is greater than or equal to, divided by, remove top) are matched longest first against a phrase table(Max munch), to ensure each one becomes a token regardless of extra spaced between words
- **Categories & symbol table:** keywords, identifiers, literals, punctuation and newlines. Each token must be categories based on the category. And identifiers to be inserted in a symbol table.

Reserved Words

Declarations: initialize, set, to, fix, define, with	**Types**: integer, float, string , bool, list nothing	**Literals:** true, false
Input / output: say, ask, into	**Arithmetic:** add subtract multiply divide mod from by	**Expression operators:, and** ,or , not

Comparison: is equal to , is not equal to is great than , is less that, is greater than or equal to , is less that or equal to	**Selection:** if , otherwise , check , when	**Loops:** repeat ,for, each in ,while ,stop skip
Functions: function, call, return ,returning	**Collections:** put ,on , remove , top , item of , length , capacity	**Objects:** public,private constructor, inherits, struct, enum , class

Parsing

Parsing is the heart of the Plainspeak compilation process. It converts the token buffer into a hierarchical structure that represents the flow of the programmes, and the design of this structure determines how well the compiler captures the programmer's intention and desire. The parser produces a unique valid syntax tree that resembles the structure of the source code.

- **Approach**: The language takes a top down recursive descent, in one single pass. The parser starts from the top level grammar rule and expands it into sub rules, with one function per rule, until the tokens match the rules definition. The grammar is context free.
- **Type**: LL(k). The input is read left to right, rules are expanded without backtracking, and k is the number of tokens of lookahead.
- **Features**: Operator precedence is handled during parsing, so "* mult" higher precedence than "+add", and control flow structures such as nested if statements are supported.
- **Limitations**: The parser does no type checking, which is left to the semantic analyser. Nesting depth is bounded by the recursion limit of llvm implementation complexity.
- **Error handling**: The parser uses panic mode recovery. It is triggered by any error token from the lexer or by any grammar rule that fails to match.
- **Output**: Either a complete AST, or a list of errors, each with its location

Syntactic / Semantic Analysis

Basic structural checks are made before semantic analysis begins: a program must start with a label, each label block must end with ;, and braces must be balanced. Semantic analysis then checks the rules that the grammar cannot express, which depend on context such declarations, types and scope. Plain speaks semantic analysis is simple but still gives broad type safety and control-flow checking. The supported checks are:

- **Strong typing:** Types are checked strictly and no implicit type promotion is allowed
- **Function checks:** Argument types and counts are checked against the function's declaration, and the returned value is checked against its declared return type
- **Scope and declarations:** All variables have to be declared before they are used. Names must be visible in the scope that they are used in
- **Valid control paths:** Every path through a function that returns a value must end in a return
- **Illegal constructs:** Statements that make no sense in context are rejected, for example break outside a loop
- **Objects:** Objects are compared by structure, a class may have only one parent, and polymorphic behaviour is not permitted

Semantic Analysis results in a types AST which is use in code gen, semantic analysis is done with multiple pass over the parsed AST each pass targets a selection of language rules checks, and result in a warning or an error panic. The final AST also resolves in language function like range and match as the target may not support such complex operation in a single operation.

Code Generation / Optimisations

The code generator walks the typed AST and makes LLVM IR which LLVM's toolchain then optimises and compiles to native code. Because Plainspeak is statically typed, every type is known by this stage, so each one maps directly to an LLVM type. Each function becomes an LLVM function. Optimisations. The compiler does not implement its own optimiser. Instead, it relies on LLVM's optimisation passes.

- **Constant folding and constant propagation**
- **Compile-time evaluation**
- **Dead code elimination**

Testing

Our three main testing methods include unit testing, integration testing and negative testing. Unit testing allows for us to test different stages of the code compilation. Token lists for the lexer and AST shape for the parser. Our integration tests will compile and run whole .psk programs to ensure that programming in our language works and ensure that the full process of tokenisation, parsing and execution run as expected. Negative tests are deliberately invalid and are designed to purposely reject .psk files to ensure that the program is successfully catching errors. Following are the types of tests that we will conduct. (Note, this is not an exhaustive list)

Lexer tests

	We give it	Expected Output
1	Say "hi"	Keyword say and identify string, value hi
2	If x is greater than 5 (	Is greater than keyword, x is identifiers ,5 is literal

Semantic Analyser Tests

	We give it	Expected Output
1	Set y to 5	Error: y is used before its declared
2	Initialize integer x Set x to "hi"	Error: test x cant store a string
3	Initialize function f() returning integer {}	Error: never returns an integer

Class unit test example	Recursion unit test example

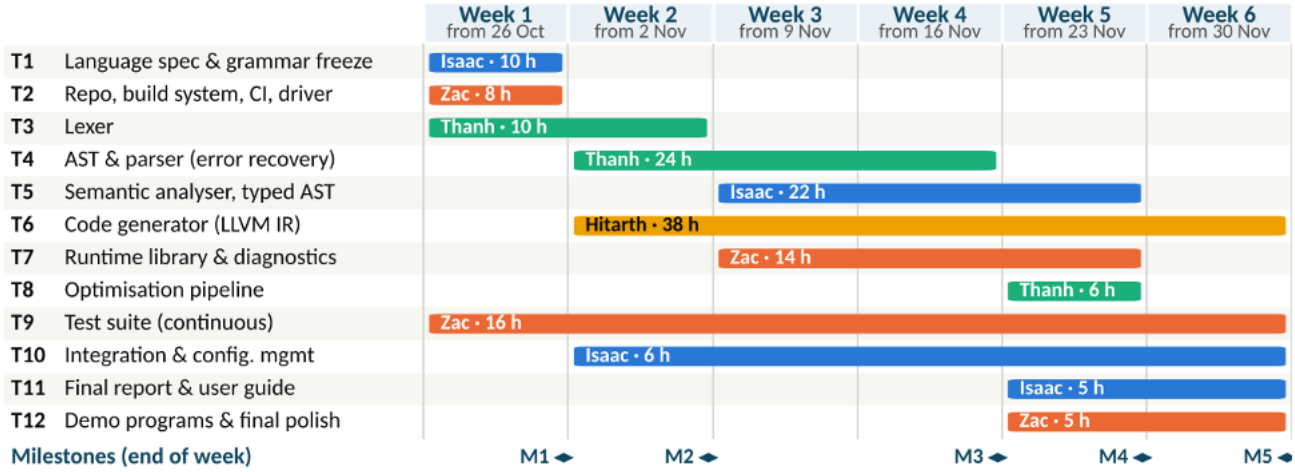
// Make a Dog object and use it
define class Dog {
public initialize string name
public define constructor (string who){
set name to who
}
public define function bark() returning nothing {
say "woof"
}
}
Initialize Dog pike with ("Pike")
call bark() from pike
say name of pike **// Should print woof, then pike**
	initialize function Foo (integer X) returning nothing {
	if X is equal to 1{
	Return nothing
	}
	subtract 1 from X
	Call Foo(X)
	Return nothing
	}
	
	Call foo(10) **// recursive 10,9... to 1**

Work Structure Breakdown

This phase uses agile methodology and is planned out to be six one week sprints that target start on Monday 26th of October. The difficulty column reflects how much responsibility each person will have. The total hours are 164, which is around 41 hours per person over the duration of the project, so around 6.5hrs per week which we decided was manageable with our other university tasks.

ID	Task	Method	Difficulty	Hours	Person
T1	Language specification and grammar freeze	EBNF grammar, keyword list, example programs	Medium	10	Isaac
T2	Repository, build system, CI, driver	make project, LLVM set up, CI that builds and runs all tests, driver	Low	8	Zac
T3	Lexer	single pass scanner, regular expressions, longest match phrases, error tokens	Low	10	Thanh
T4	AST and parser	recursive descent, LL(k), precedence climbing	High	24	Thanh
T5	Semantic analyser and typed AST	declarations, scope stack and types, control flow	High	22	Isaac
T6	Code generator	typed AST to LLVM IR with IR Builder Code gen	High	38	Hitarth
T7	Runtime library and diagnostics	C++ runtime for say/ask, run time checks	Medium	14	Zac
T8	Optimisation pipeline	LLVM pass manager	Low	6	Thanh
T9	Test suite	unit tests per stage, negative tests etc	Medium	16	Zac
T10	Integration and configuration management	pull request review and merge	Medium	6	Isaac
T11	Final report and user guide	update this report to match the implementation	Low	5	Isaac
T12	Demo programs and final polish	five showcase programs	Low	5	Zac

On a gantt chart it looks like the following:



Gant chart of timeline created using AI tool

Roles and Responsibilities

Person	Responsibility	Hours	Report Section Completed
Isaac	Project lead and meeting organiser. Language specification and grammar freeze (T1). Semantic analyser (T5), integration and configuration management (T10), final report helper (T11)	43	Introduction, basic syntax, work structure breakdown, roles
Zac	Repository maker, build and CI (T2), (T7), test stuff (T9), and demo maker (T12)	43	advanced syntax and testing
Thanh	lexer (T3), AST and parser (T4), optimisation pipeline (T8), Code generator	40	compiler Structure, tokenisation and lexical analysis, parsing
Hitarth	Code generator (T6), reviews all semantic analysis and optimisation changes	38	compiler structure, syntactic and semantic analysis, code generation, optimisations